FOR COMPLETE BEGINNERS

Same values. Two objects.

Complete Modern Java, episode 21: removing from the cart, in Java and TypeScript

> Why Java said Milk was not in the cart, and how to remove by a rule you choose.

remove(new Product("Milk", 99)) returned false, with Milk right there in the cart.

new makes a new object. Same values, two objects, and Product never said what equal means.

In this episode

- remove returns a boolean
- Same values, two objects
- == and equals on objects
- Removing the same object
- removeByName
- TypeScript indexOf and splice

Coming next

- HashMap: find a product by name straight away (episode 22)

At the time of recording: Java 25, Node 24 LTS, IntelliJ IDEA 2026.1.3.

01 What each run printed

remove(new Product("Milk", 99))	`false; count stays 3`
first == second, first.equals(second)	`false, false`
"Milk".equals("Milk")	`true`
remove(milk), the same object	`true; count 2, total 400`
removeByName("Eggs"), removeByName("Rice")	`true, false; total 150`
returns.ts: indexOf(second)	`-1`

Every row was run, not predicted.

02 Why Java said no

remove asks each item whether it is equal. Product has never said what equal means, so only the very same object counts.

Strings have that rule: same text, equal. That is why removing "Milk" from a list of strings worked.

Twins.java

```
void main() {
    Product first = new Product("Milk", 99);
    Product second = new Product("Milk", 99);
    IO.println(first == second);          // false
    IO.println(first.equals(second));     // false
    IO.println("Milk".equals("Milk"));    // true
}
```

> Printed false, false, true.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

03 A rule of our own: remove by name

Remove the first product whose name matches, and stop straight away. Matching is exact: "milk" is not "Milk".

Real shops identify products by a code; for this small cart, the name is the rule we chose.

ShoppingCart.java

```
boolean remove(Product product) {
    return items.remove(product);
}

boolean removeByName(String name) {
    for (int i = 0; i < items.size(); i++) {
        if (items.get(i).getName().equals(name)) {
            items.remove(i);
            return true;
        }
    }
    return false;
}
```

> Returns.java printed false, 3, true, 2, 400, true, false, 150.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

04 TypeScript: the same idea

=== on two objects is only true for the very same object, so indexOf(second) is -1. Search by a value, then splice it out.

returns.ts

```
function removeByName(items: Product[], name: string): boolean {
    for (let i = 0; i < items.length; i++) {
        if (items[i].name === name) {
            items.splice(i, 1);
            return true;
        }
    }
    return false;
}
```

> node printed false, -1, true, 1.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

Same or equal?

1. new always makes a new object.
2. == on objects: the same object?
3. Strings: compare with equals.
4. Want a value match? Write the rule yourself.
5. Challenge: Milk, Eggs, Milk, Bread; removeByName("Milk") once. What is left?

Everything else

Site
https://code-with-sam-dev.github.io

YouTube
https://www.youtube.com/@CodewithSam-Dev

LinkedIn
https://www.linkedin.com/company/code-with-sam-dev

X
https://x.com/CodeWithSamDev

TikTok
https://www.tiktok.com/@codewithsamdev

Instagram
https://www.instagram.com/codewithsamdev

Facebook
https://www.facebook.com/codewithsamdev

GitHub
https://github.com/code-with-sam-dev

Buy me a coffee
https://buymeacoffee.com/codewithsam

Buy me sadza
https://buymesadza.com/codewithsam

The code and the challenge files: complete-modern-java
https://github.com/code-with-sam-dev/complete-modern-java

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. TypeScript is a trademark of Microsoft. IntelliJ IDEA is a trademark of JetBrains. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-11-07 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.